pipeline

March 19, 2025

Elements of workflow adapted from https://github.com/eu-cdse/notebook-samples/blob/main/openeo/NDVI_Timeseries.ipynb

[1]:
```
import os
import json

experiment_name = "pacific_palisades"
experiments_dir = "experiments"
experiment_path = os.path.join(experiments_dir, experiment_name)
os.makedirs(experiment_path, exist_ok=True)

SPATIAL_EXTENT = {"west": -118.5650, "south": 34.0300, "east": -118.5000,␣
 ↪"north": 34.0900}

TEMPORAL_EXTENT = ["2024-10-01", "2025-02-28"]
TEMPORAL_EXTENT_FIRE = ["2025-01-07", "2025-01-31"]
TEMPORAL_EXTENT_PRE_FIRE = [TEMPORAL_EXTENT[0], TEMPORAL_EXTENT_FIRE[0]]
TEMPORAL_EXTENT_POST_FIRE = [TEMPORAL_EXTENT_FIRE[1], TEMPORAL_EXTENT[1]]


coordinates = [
    [SPATIAL_EXTENT["west"], SPATIAL_EXTENT["north"]],
    [SPATIAL_EXTENT["east"], SPATIAL_EXTENT["north"]],
    [SPATIAL_EXTENT["east"], SPATIAL_EXTENT["south"]],
    [SPATIAL_EXTENT["west"], SPATIAL_EXTENT["south"]],
    [SPATIAL_EXTENT["west"], SPATIAL_EXTENT["north"]]
]

# Construct the GeoJSON object
fields = {
    "type": "FeatureCollection",
    "features": [
        {
            "type": "Feature",
            "properties": {
                "temporal_extent": TEMPORAL_EXTENT
            },
            "geometry": {
```

```
                "type": "Polygon",
                "coordinates": [coordinates]
            }
        }
    ]
}

fields_path = os.path.join(experiment_path, "fields.json")
with open(fields_path, "w") as f:
    json.dump(fields, f, indent=4)

print(f"GeoJSON saved at: {fields_path}")

fields
```

```
GeoJSON saved at: experiments/pacific_palisades/fields.json
```

[1]:
```
{'type': 'FeatureCollection',
 'features': [{'type': 'Feature',
   'properties': {'temporal_extent': ['2024-10-01', '2025-02-28']},
   'geometry': {'type': 'Polygon',
    'coordinates': [[[-118.565, 34.09],
      [-118.5, 34.09],
      [-118.5, 34.03],
      [-118.565, 34.03],
      [-118.565, 34.09]]]}}]}
```

[6]:
```
from utils.openeo_connection import get_openeo_connection

connection = get_openeo_connection()

ndvi_csv_path = os.path.join(experiment_path, "ndvi-results",␣
 ↪"timeseries-smoothed.csv")
s5p_csv_path = os.path.join(experiment_path, "s5p-results", "s5p_fire.csv")
```

```
Authenticated using refresh token.
```

0.1 Sentinel-2

[3]:
```
s2cube = connection.load_collection(
    "SENTINEL2_L2A",
    temporal_extent = TEMPORAL_EXTENT,
    bands=["B04", "B08", "SCL"]
)
```

0.1.1 NDVI

[4]:
```
import scipy.signal
import numpy as np
import pandas as pd

# Compute NDVI
red = s2cube.band("B04")
nir = s2cube.band("B08")
ndvi = (nir - red) / (nir + red)

# Cloud mask using the SCL band
scl = s2cube.band("SCL")
mask = ~((scl == 4) | (scl == 5))

# 2D Gaussian kernel
g = scipy.signal.windows.gaussian(11, std=1.6)
kernel = np.outer(g, g)
kernel = kernel / kernel.sum()

# Morphological dilation of mask: convolution + threshold
mask = mask.apply_kernel(kernel)
mask = mask > 0.1

# Apply mask to NDVI
ndvi_masked = ndvi.mask(mask)
timeseries_masked = ndvi_masked.aggregate_spatial(geometries=fields,␣
 ↪reducer="mean")

# Convert to Pandas & Save
timeseries_masked_df = pd.DataFrame(timeseries_masked.execute())
masked_output_csv = os.path.join(experiment_path, "timeseries_masked.csv")
timeseries_masked_df.to_csv(masked_output_csv, index=False)

print(f"Masked timeseries saved at: {masked_output_csv}")
timeseries_masked
```

```
Masked timeseries saved at: experiments/pacific_palisades/timeseries_masked.csv
```

[4]:
```
<openeo.rest.vectorcube.VectorCube at 0x11f95ee40>
```

[ ]:
```
import openeo

# Define UDF for Savitzky-Golay smoothing
udf = openeo.UDF(
    """
from scipy.signal import savgol_filter
from openeo.udf import XarrayDataCube
```

```
import numpy as np
import xarray as xr

def apply_datacube(cube: XarrayDataCube, context: dict) -> XarrayDataCube:
    array = cube.get_array()

    # Get the number of available time points
    time_steps = array.sizes["t"]

    # Ensure at least 2 points for interpolation and 3 for smoothing
    if time_steps < 2:
        return cube  # Not enough points for interpolation
    if time_steps < 3:
        return XarrayDataCube(array.fillna(0))  # Fill missing with 0 if␣
 ↪smoothing isn't possible

    # Fill NaN values using forward fill, then backward fill
    filled = array.ffill(dim='t').bfill(dim='t')

    # Determine a safe window length (must be odd and <= number of time points)
    window_size = min(time_steps, 5)  # Do not exceed available points
    if window_size % 2 == 0:
        window_size -= 1  # Ensure it's an odd number

    # Apply Savitzky-Golay smoothing
    smoothed_array = savgol_filter(filled.values, window_length=window_size,␣
 ↪polyorder=2, axis=0)

    # Return smoothed data as a new XarrayDataCube
    return XarrayDataCube(xr.DataArray(smoothed_array, dims=array.dims,␣
 ↪coords=array.coords))
    """
)

# Apply smoothing function
ndvi_smoothed = ndvi_masked.apply_dimension(code=udf, dimension="t")

# Aggregate smoothed NDVI over spatial extent
timeseries_smoothed = ndvi_smoothed.aggregate_spatial(geometries=fields,␣
 ↪reducer="mean")

# Convert to Pandas & Save
timeseries_smoothed_df = pd.DataFrame(timeseries_smoothed.execute())
smoothed_output_csv = os.path.join(experiment_path, "timeseries_smoothed.csv")
timeseries_smoothed_df.to_csv(smoothed_output_csv, index=False)

print(f"Smoothed timeseries saved at: {smoothed_output_csv}")
```

```
timeseries_smoothed
```

```
Smoothed timeseries saved at:
experiments/pacific_palisades/timeseries_smoothed.csv
```

[ ]:
```
<openeo.rest.vectorcube.VectorCube at 0x12a635590>
```

[ ]:
```
results_dir = os.path.join(experiment_path, "ndvi-results")
os.makedirs(results_dir, exist_ok=True)
csv_path = os.path.join(results_dir, "timeseries-smoothed.csv")

if os.path.exists(csv_path):
    print(f"Skipping execution: File already exists at {csv_path}")
else:
    print("Executing batch processing...")

    # Execute batch processing
    job = timeseries_smoothed.execute_batch(
        out_format="CSV", title="Smoothed NDVI timeseries"
    )

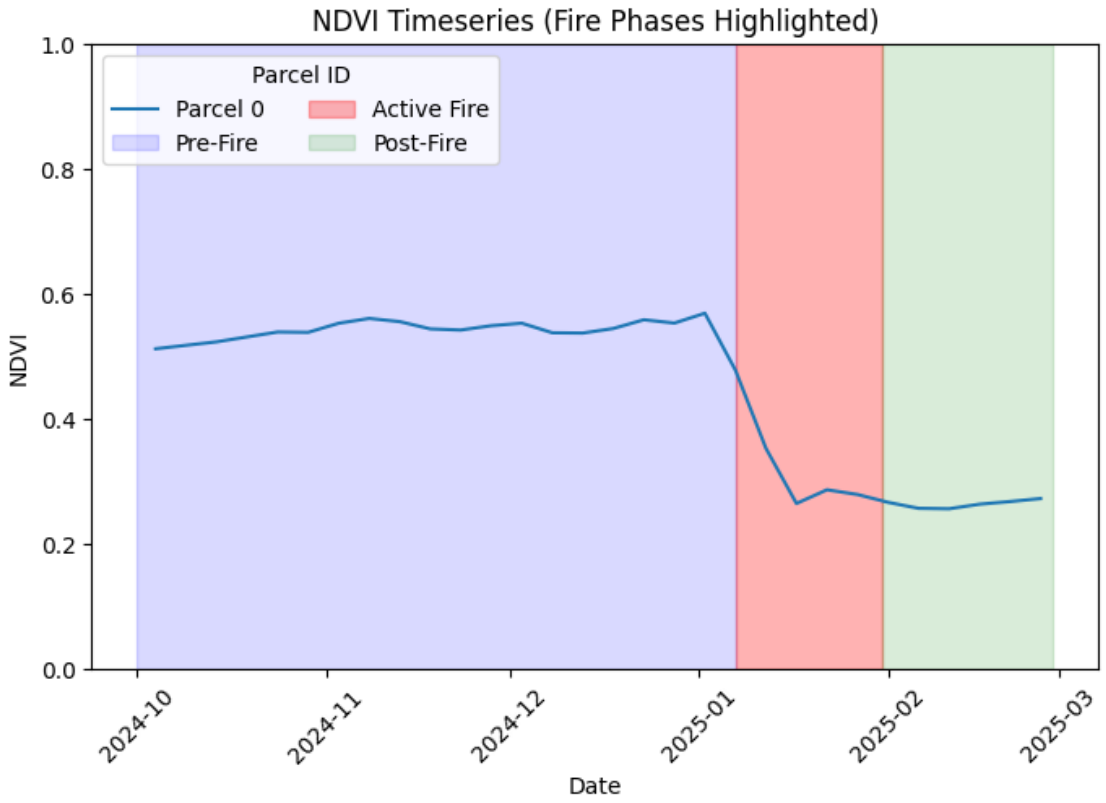
    job.get_results().download_file(csv_path)

    print(f"Results saved at: {csv_path}")
```

```
Executing batch processing…
0:00:00 Job 'j-2503190959064fa9a89fd30efdb9649f': send 'start'
0:00:14 Job 'j-2503190959064fa9a89fd30efdb9649f': created (progress 0%)
0:00:19 Job 'j-2503190959064fa9a89fd30efdb9649f': created (progress 0%)
0:00:26 Job 'j-2503190959064fa9a89fd30efdb9649f': created (progress 0%)
0:00:34 Job 'j-2503190959064fa9a89fd30efdb9649f': created (progress 0%)
0:00:44 Job 'j-2503190959064fa9a89fd30efdb9649f': created (progress 0%)
0:00:56 Job 'j-2503190959064fa9a89fd30efdb9649f': created (progress 0%)
0:01:12 Job 'j-2503190959064fa9a89fd30efdb9649f': running (progress N/A)
0:01:31 Job 'j-2503190959064fa9a89fd30efdb9649f': running (progress N/A)
0:01:55 Job 'j-2503190959064fa9a89fd30efdb9649f': running (progress N/A)
0:02:26 Job 'j-2503190959064fa9a89fd30efdb9649f': running (progress N/A)
0:03:03 Job 'j-2503190959064fa9a89fd30efdb9649f': running (progress N/A)
0:03:50 Job 'j-2503190959064fa9a89fd30efdb9649f': finished (progress 100%)
Results saved at: experiments/pacific_palisades/ndvi-results/timeseries-
smoothed.csv
```

[7]:
```
from bin.plot_timeseries import plot_ndvi_timeseries

plot_ndvi_timeseries(ndvi_csv_path, experiment_path, TEMPORAL_EXTENT_PRE_FIRE,␣
↪TEMPORAL_EXTENT_FIRE, TEMPORAL_EXTENT_POST_FIRE)
```



Plot saved at: experiments/pacific_palisades/plots/ndvi_fire_phases.png

0.2 Sentinel-5p

[ ]:
```
import os
import pandas as pd

# List of Sentinel-5P bands to process
s5p_bands = ["CO", "HCHO", "NO2", "O3", "SO2", "CH4"]

results_dir = os.path.join(experiment_path, "s5p-results")
os.makedirs(results_dir, exist_ok=True)
s5p_output_csv = os.path.join(results_dir, "s5p_fire.csv")


if os.path.exists(s5p_output_csv):
    print(f"Skipping execution: File already exists at {s5p_output_csv}")
else:
    print("Processing Sentinel-5P data...")

    # Dictionary dataFrames for each band
```

```
    band_dfs = {}

    # Function to process each band and convert to DataFrame
    def process_s5p_band(band_name):
        """Loads Sentinel-5P data for a specific band, aggregates, and returns a cleaned DataFrame."""
        print(f"Processing {band_name} data...")

        # Load Sentinel-5P L2 data for the band
        s5cube = connection.load_collection(
            "SENTINEL_5P_L2",
            temporal_extent=TEMPORAL_EXTENT,
            bands=[band_name]  # Can only load one band at a time
        )

        # Aggregate temporally (daily means)
        s5fire = s5cube.aggregate_temporal_period(reducer="mean", period="day")

        # Aggregate spatially (mean per parcel)
        s5fire = s5fire.aggregate_spatial(reducer="mean", geometries=fields)

        # Execute the processing graph and retrieve the result
        s5fire_result = s5fire.execute()

        df = pd.DataFrame(s5fire_result.items(), columns=["date", band_name])
        df["date"] = pd.to_datetime(df["date"])

        df[band_name] = df[band_name].apply(
            lambda x: x[0][0] if isinstance(x, list) and x and isinstance(x[0], list) and x[0] else None
        )

        df[band_name] = pd.to_numeric(df[band_name], errors="coerce")

        return df

    for band in s5p_bands:
        band_dfs[band] = process_s5p_band(band)

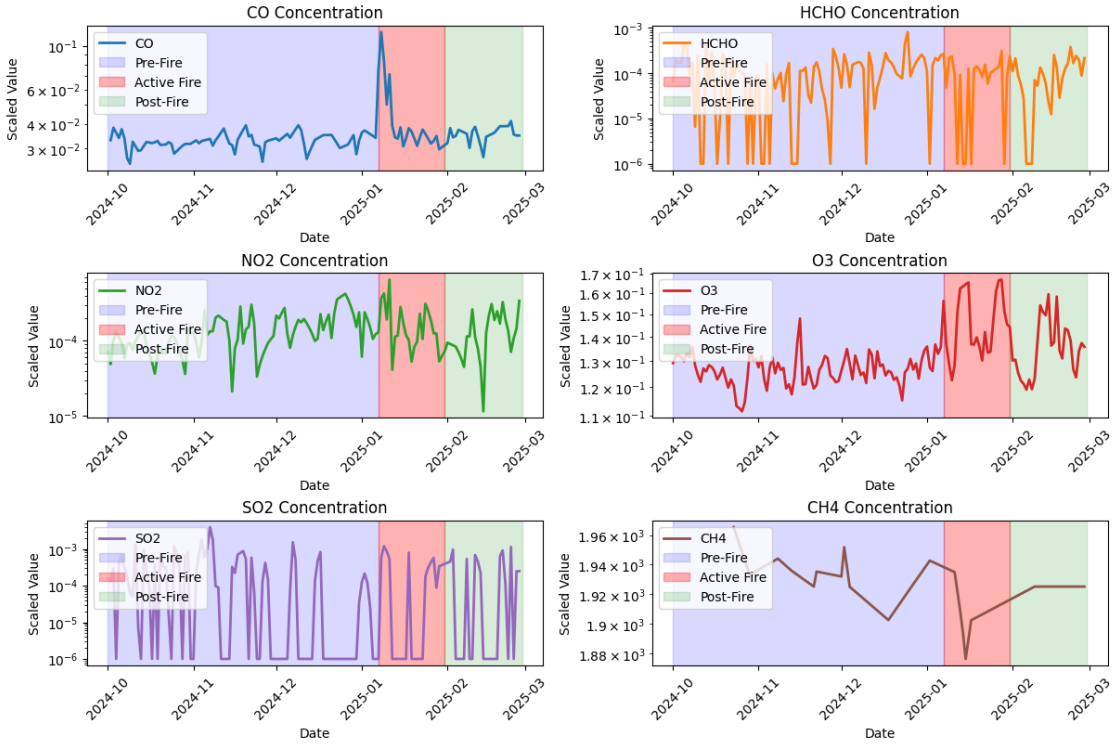
    s5p_combined_df = pd.DataFrame({"date": band_dfs[s5p_bands[0]]["date"]})
    for band in s5p_bands:
        s5p_combined_df = pd.merge(s5p_combined_df, band_dfs[band], on="date", how="outer")

    # Save merged results as CSV
    s5p_combined_df.to_csv(s5p_output_csv, index=False)
    print(f"S5P fire data {s5p_bands} saved at: {s5p_output_csv}")
```

```
Processing Sentinel-5P data…
Processing CO data…
Processing HCHO data…
Processing NO2 data…
Processing O3 data…
Processing SO2 data…
Processing CH4 data…
S5P fire data ['CO', 'HCHO', 'NO2', 'O3', 'SO2', 'CH4'] saved at:
experiments/pacific_palisades/s5p-results/s5p_fire.csv
```

[9]:
```
from bin.plot_timeseries import plot_s5p_timeseries

plot_s5p_timeseries(s5p_csv_path, experiment_path, TEMPORAL_EXTENT_PRE_FIRE,␣
↪TEMPORAL_EXTENT_FIRE, TEMPORAL_EXTENT_POST_FIRE)
```




```
Plot saved at: experiments/pacific_palisades/plots/s5p_fire_phases.png
```

0.3 Merging

[ ]:
```
pre_fire_start, pre_fire_end = map(lambda d: pd.to_datetime(d).
↪tz_localize(None), TEMPORAL_EXTENT_PRE_FIRE)
fire_start, fire_end = map(lambda d: pd.to_datetime(d).tz_localize(None),␣
↪TEMPORAL_EXTENT_FIRE)
```

```
post_fire_start, post_fire_end = map(lambda d: pd.to_datetime(d).
  ↪tz_localize(None), TEMPORAL_EXTENT_POST_FIRE)


results_dir = os.path.join(experiment_path, "merged-results")
os.makedirs(results_dir, exist_ok=True)

# Load NDVI data
ndvi_df = pd.read_csv(ndvi_csv_path)
ndvi_df.rename(columns={"band_unnamed": "ndvi"}, inplace=True)

# Load Sentinel-5P data
s5p_df = pd.read_csv(s5p_csv_path)

# Convert date columns to make timezone-naive
ndvi_df["date"] = pd.to_datetime(ndvi_df["date"]).dt.tz_localize(None)
s5p_df["date"] = pd.to_datetime(s5p_df["date"]).dt.tz_localize(None)

# Merge NDVI and S5P DataFrames on the "date" column outer join
merged_df = pd.merge(ndvi_df, s5p_df, on="date", how="outer")

# Assign phase labels
def categorize_phase(date):
    if pd.isnull(date):
        return None
    if pre_fire_start <= date < pre_fire_end:
        return "pre"
    elif fire_start <= date <= fire_end:
        return "active"
    elif post_fire_start < date <= post_fire_end:
        return "post"
    return None

merged_df["phase"] = merged_df["date"].apply(categorize_phase)

merged_df.drop(columns=["feature_index"], inplace=True)

merged_csv_path = os.path.join(results_dir, "merged_timeseries.csv")
merged_df.to_csv(merged_csv_path, index=False)

print(f"Merged timeseries saved at: {merged_csv_path}")

merged_df
```

```
Merged timeseries saved at: experiments/pacific_palisades/merged-
results/merged_timeseries.csv
```

[ ]:
```
           date      ndvi        CO      HCHO       NO2        O3       SO2  \
0    2024-09-30       NaN       NaN       NaN       NaN       NaN       NaN
1    2024-10-01       NaN       NaN  0.000065  0.000071  0.129199  0.000158
2    2024-10-02       NaN  0.033073  0.000191  0.000049  0.132006  0.000129
3    2024-10-03       NaN  0.038286  0.000172  0.000102  0.132279  0.000295
4    2024-10-04  0.512399       NaN  0.000170  0.000131  0.131550 -0.001080
..          …       …         …         …         …         …
146  2025-02-23       NaN  0.039009  0.000167  0.000136  0.126650 -0.000693
147  2025-02-24       NaN  0.041395  0.000242  0.000071  0.123703  0.001138
148  2025-02-25       NaN  0.035377  0.000199  0.000109  0.133902 -0.000219
149  2025-02-26  0.272753  0.034939  0.000089  0.000146  0.137364  0.000248
150  2025-02-27       NaN       NaN  0.000216  0.000340  0.135785       NaN

     CH4 phase
0    NaN  None
1    NaN   pre
2    NaN   pre
3    NaN   pre
4    NaN   pre
..   …    …
146  NaN  post
147  NaN  post
148  NaN  post
149  NaN  post
150  NaN  post

[151 rows x 9 columns]
```

0.4 Model

[ ]:
```
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler

df = pd.read_csv(merged_csv_path)

numeric_columns = ["ndvi", "CO", "HCHO", "NO2", "O3", "SO2", "CH4"]

df[numeric_columns] = df[numeric_columns].interpolate(method="linear")
df[numeric_columns] = df[numeric_columns].ffill()
df[numeric_columns] = df[numeric_columns].bfill()
```

```
scaler = StandardScaler()
df[numeric_columns] = scaler.fit_transform(df[numeric_columns])

class TimeSeriesWindowDataset(Dataset):
    def __init__(self, data, window_size):
        """
        data: numpy array of shape (time_steps, num_features)
        window_size: int, the length of each time window
        """
        self.data = data
        self.window_size = window_size

    def __len__(self):
        return len(self.data) - self.window_size + 1

    def __getitem__(self, idx):
        # Extract a window of shape (window_size, num_features)
        window = self.data[idx: idx + self.window_size]
        return torch.tensor(window.T, dtype=torch.float)

data_array = df[numeric_columns].values
window_size = 10
dataset = TimeSeriesWindowDataset(data_array, window_size)
dataloader = DataLoader(dataset, batch_size=32, shuffle=True)
```

[28]:
```
from model.vq_vae import VQVAE

# Model hyperparameters
input_channels = len(numeric_columns)
hidden_dim = 256
latent_dim = 64
num_embeddings = 512
commitment_cost = 0.25

model = VQVAE(input_channels, hidden_dim, latent_dim, num_embeddings,␣
 ↪commitment_cost)
optimizer = optim.Adam(model.parameters(), lr=1e-3)
mse_loss = nn.MSELoss()
```

[30]:
```
import matplotlib.pyplot as plt

num_epochs = 5000

loss_history = []

for epoch in range(num_epochs):
    model.train()
```

```
    total_loss = 0
    for batch in dataloader:
        optimizer.zero_grad()

        # Forward pass
        x_recon, vq_loss = model(batch)

        # Compute losses
        recon_loss = mse_loss(x_recon, batch)
        loss = recon_loss + vq_loss

        # Backward pass
        loss.backward()
        optimizer.step()

        total_loss += loss.item()

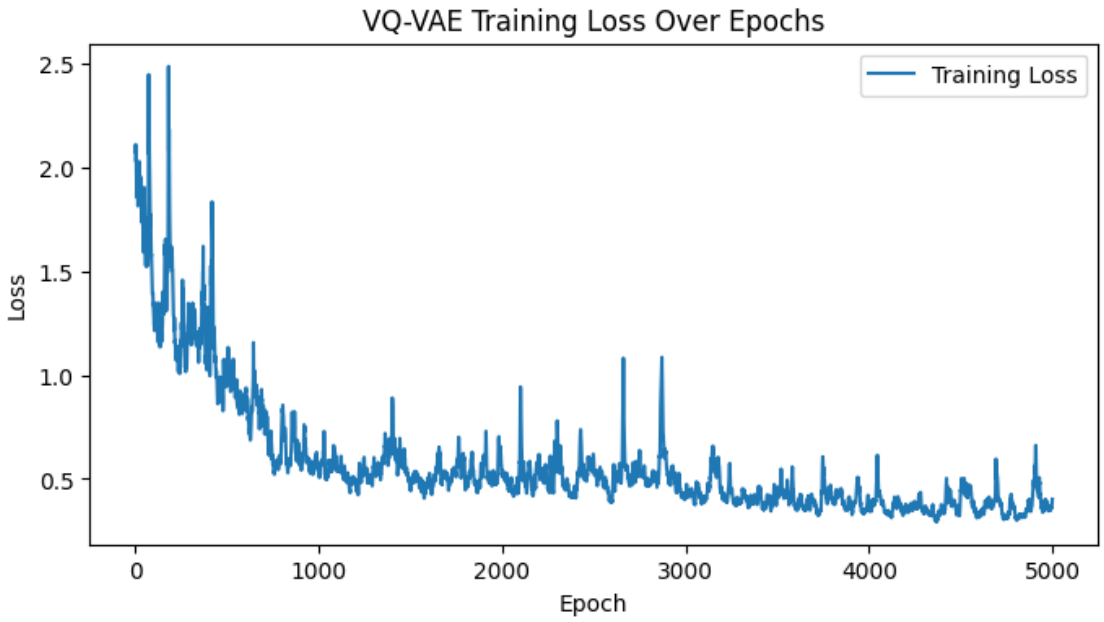
    avg_loss = total_loss / len(dataloader)
    loss_history.append(avg_loss)

    #  reduce console spam
    if (epoch + 1) % 100 == 0:
        print(f"Epoch {epoch+1}/{num_epochs}, Loss: {avg_loss:.4f}")

plt.figure(figsize=(8, 4))
plt.plot(loss_history, label="Training Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("VQ-VAE Training Loss Over Epochs")
plt.legend()
plt.show()
```

```
Epoch 100/5000, Loss: 1.3001
Epoch 200/5000, Loss: 1.5909
Epoch 300/5000, Loss: 1.2923
Epoch 400/5000, Loss: 1.2461
Epoch 500/5000, Loss: 0.9454
Epoch 600/5000, Loss: 0.8790
Epoch 700/5000, Loss: 0.8378
Epoch 800/5000, Loss: 0.7804
Epoch 900/5000, Loss: 0.5928
Epoch 1000/5000, Loss: 0.5462
Epoch 1100/5000, Loss: 0.5374
Epoch 1200/5000, Loss: 0.4809
Epoch 1300/5000, Loss: 0.5784
Epoch 1400/5000, Loss: 0.8759
Epoch 1500/5000, Loss: 0.4465
Epoch 1600/5000, Loss: 0.4743
```

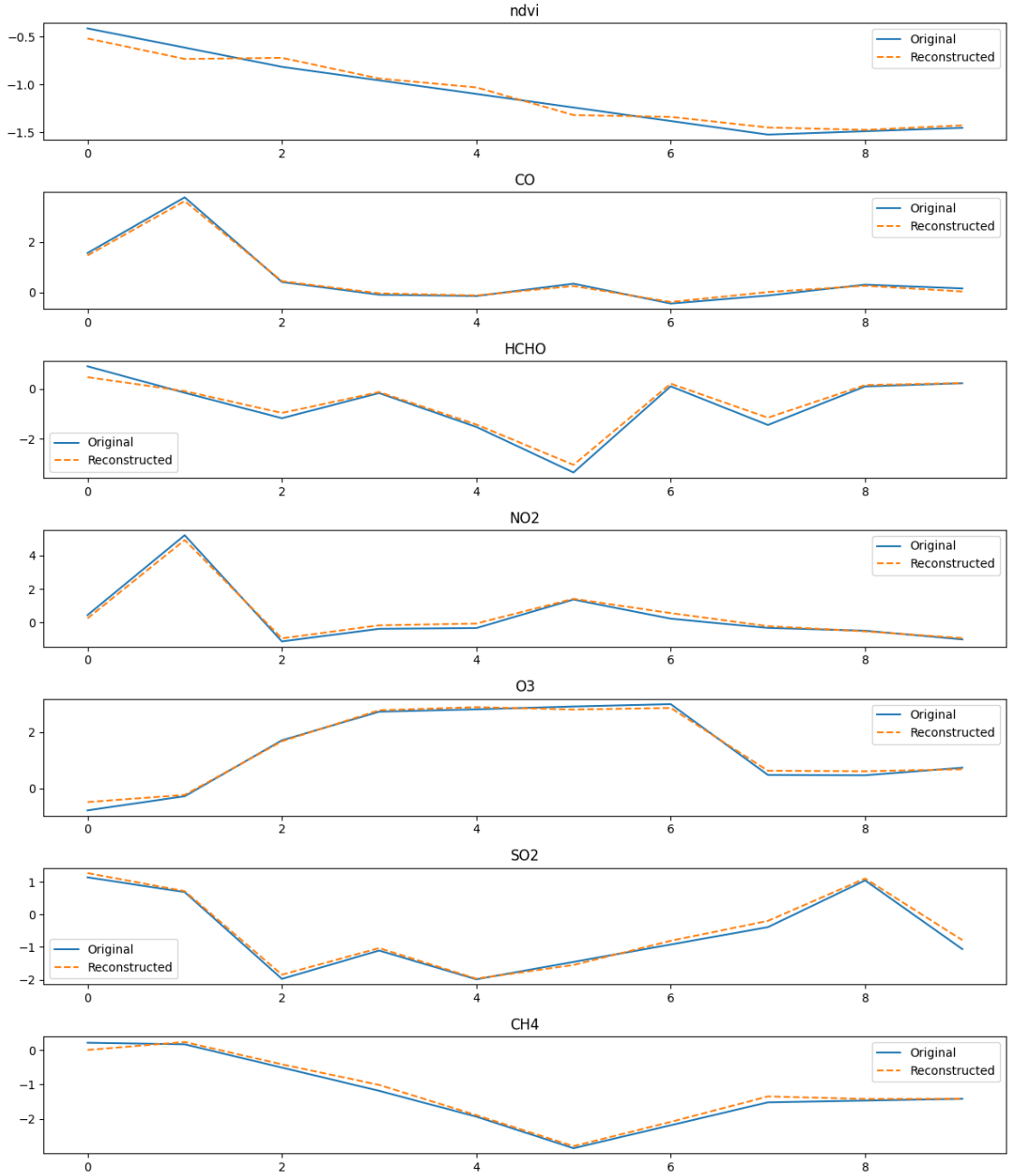
```
Epoch 1700/5000, Loss: 0.5229
Epoch 1800/5000, Loss: 0.4459
Epoch 1900/5000, Loss: 0.6152
Epoch 2000/5000, Loss: 0.5114
Epoch 2100/5000, Loss: 0.9435
Epoch 2200/5000, Loss: 0.6003
Epoch 2300/5000, Loss: 0.7797
Epoch 2400/5000, Loss: 0.4210
Epoch 2500/5000, Loss: 0.5784
Epoch 2600/5000, Loss: 0.3910
Epoch 2700/5000, Loss: 0.5197
Epoch 2800/5000, Loss: 0.4687
Epoch 2900/5000, Loss: 0.5280
Epoch 3000/5000, Loss: 0.4197
Epoch 3100/5000, Loss: 0.4179
Epoch 3200/5000, Loss: 0.4298
Epoch 3300/5000, Loss: 0.3834
Epoch 3400/5000, Loss: 0.4596
Epoch 3500/5000, Loss: 0.4529
Epoch 3600/5000, Loss: 0.3679
Epoch 3700/5000, Loss: 0.3829
Epoch 3800/5000, Loss: 0.4054
Epoch 3900/5000, Loss: 0.3742
Epoch 4000/5000, Loss: 0.3691
Epoch 4100/5000, Loss: 0.3266
Epoch 4200/5000, Loss: 0.3782
Epoch 4300/5000, Loss: 0.3360
Epoch 4400/5000, Loss: 0.3359
Epoch 4500/5000, Loss: 0.4474
Epoch 4600/5000, Loss: 0.3358
Epoch 4700/5000, Loss: 0.5033
Epoch 4800/5000, Loss: 0.3072
Epoch 4900/5000, Loss: 0.4737
Epoch 5000/5000, Loss: 0.4017
```



[32]:
```
model.eval()
with torch.no_grad():
    # Take batch from the dataloader
    batch = next(iter(dataloader))
    x_recon, _ = model(batch)

# Plot the original and reconstructed signals for each channel of the first␣
 ↪sample
sample_idx = 0
original = batch[sample_idx].cpu().numpy()      # shape: (input_channels,␣
 ↪window_size)
reconstructed = x_recon[sample_idx].cpu().numpy()

plt.figure(figsize=(12, 2 * input_channels))
for i in range(input_channels):
    plt.subplot(input_channels, 1, i+1)
    plt.plot(original[i], label="Original")
    plt.plot(reconstructed[i], label="Reconstructed", linestyle="--")
    plt.title(numeric_columns[i])
    plt.legend()
plt.tight_layout()
plt.show()
```



0.5 Prediction

[ ]:
```
import pandas as pd
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
```

```
from torch.utils.data import Dataset, DataLoader, Subset
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import (
    classification_report,
    confusion_matrix,
    f1_score,
    accuracy_score,
    ConfusionMatrixDisplay
)
import matplotlib.pyplot as plt


# 1. Load & Preprocess Data

numeric_columns = ["ndvi", "CO", "HCHO", "NO2", "O3", "SO2", "CH4"]
df[numeric_columns] = df[numeric_columns].interpolate(method="linear")
df[numeric_columns] = df[numeric_columns].ffill()
df[numeric_columns] = df[numeric_columns].bfill()

# Filter out unexpected phases, keep only pre/active/post
df = df[df["phase"].isin(["pre", "active", "post"])]

# Encode phase as integer labels
le = LabelEncoder()
df["phase_label"] = le.fit_transform(df["phase"])  # 0,1,2 => pre, active, post

# 2. Create a Windowed Dataset

class PhaseWindowDataset(Dataset):
    def __init__(self, df, numeric_cols, window_size=10):
        """
        df: pandas DataFrame containing numeric_cols and 'phase_label'
        numeric_cols: list of numeric feature names
        window_size: int, length of each time window
        """
        self.df = df.reset_index(drop=True)
        self.numeric_cols = numeric_cols
        self.window_size = window_size

        self.values = self.df[numeric_cols].values
        self.phases = self.df["phase_label"].values

    def __len__(self):
        return len(self.df) - self.window_size + 1

    def __getitem__(self, idx):
```

```
        window_data = self.values[idx : idx + self.window_size]
        # Label is from the last timestep in this window
        label = self.phases[idx + self.window_size - 1]

        # For 1D CNN: (num_features, window_size)
        window_data = torch.tensor(window_data.T, dtype=torch.float)
        label = torch.tensor(label, dtype=torch.long)

        return window_data, label

dataset = PhaseWindowDataset(df, numeric_columns, window_size=10)

# 3. Define Model & Helpers

class CNNPhaseClassifier(nn.Module):
    def __init__(self, input_channels, num_classes):
        super(CNNPhaseClassifier, self).__init__()
        self.conv1 = nn.Conv1d(input_channels, 64, kernel_size=3, padding=1)
        self.conv2 = nn.Conv1d(64, 128, kernel_size=3, padding=1)
        self.pool = nn.AdaptiveAvgPool1d(1)  # global average pooling
        self.relu = nn.ReLU()
        self.fc = nn.Linear(128, num_classes)

    def forward(self, x):
        # x: (batch_size, input_channels, window_size)
        x = self.relu(self.conv1(x))
        x = self.relu(self.conv2(x))
        x = self.pool(x)
        x = x.squeeze(-1)
        x = self.fc(x)
        return x

def train_one_epoch(model, dataloader, criterion, optimizer):
    model.train()
    total_loss = 0
    for batch_data, batch_labels in dataloader:
        optimizer.zero_grad()
        outputs = model(batch_data)
        loss = criterion(outputs, batch_labels)
        loss.backward()
        optimizer.step()
        total_loss += loss.item()
    return total_loss / len(dataloader)

def evaluate_model(model, dataloader):
    model.eval()
    all_preds = []
```

```
    all_labels = []
    with torch.no_grad():
        for batch_data, batch_labels in dataloader:
            outputs = model(batch_data)
            preds = torch.argmax(outputs, dim=1)
            all_preds.extend(preds.cpu().numpy())
            all_labels.extend(batch_labels.cpu().numpy())
    return np.array(all_labels), np.array(all_preds)

def plot_confusion_and_report(true_labels, pred_labels, label_encoder,␣
 ↪title=""):
    """
    Plots confusion matrix and prints classification report for all 3 classes
    (pre, active, post), even if some classes are missing in the test set.
    """
    all_labels = [0, 1, 2]
    cm = confusion_matrix(true_labels, pred_labels, labels=all_labels)
    disp = ConfusionMatrixDisplay(confusion_matrix=cm,␣
 ↪display_labels=label_encoder.classes_)
    disp.plot(cmap=plt.cm.Blues)
    plt.title(f"Confusion Matrix: {title}")
    plt.show()

    print(f"\nClassification Report: {title}")
    print(classification_report(
        true_labels,
        pred_labels,
        labels=all_labels,
        target_names=label_encoder.classes_
    ))

# 4a. K-Fold Cross Validation
def cross_validate_model(dataset, k=5, epochs=10):
    from sklearn.model_selection import StratifiedKFold

    X_indices = np.arange(len(dataset))
    y_labels = []
    for idx in range(len(dataset)):
        _, label = dataset[idx]
        y_labels.append(label.item())
    y_labels = np.array(y_labels)

    skf = StratifiedKFold(n_splits=k, shuffle=True, random_state=42)

    fold_accuracies = []
    fold_f1_scores = []
```

```
    for fold, (train_idx, val_idx) in enumerate(skf.split(X_indices, y_labels)):
        print(f"\n=== K-Fold: Fold {fold+1}/{k} ===")

        train_subset = Subset(dataset, train_idx)
        val_subset   = Subset(dataset, val_idx)

        train_loader = DataLoader(train_subset, batch_size=32, shuffle=True)
        val_loader   = DataLoader(val_subset, batch_size=32, shuffle=False)

        # New model each fold
        model = CNNPhaseClassifier(input_channels=len(numeric_columns),␣
 ↪num_classes=3)
        criterion = nn.CrossEntropyLoss()
        optimizer = optim.Adam(model.parameters(), lr=1e-3)

        # Train
        for epoch in range(epochs):
            loss = train_one_epoch(model, train_loader, criterion, optimizer)

        # Evaluate
        val_labels, val_preds = evaluate_model(model, val_loader)
        acc = accuracy_score(val_labels, val_preds)
        f1 = f1_score(val_labels, val_preds, average="weighted")

        print(f"Fold {fold+1} - Accuracy: {acc:.4f}, Weighted F1: {f1:.4f}")

        fold_accuracies.append(acc)
        fold_f1_scores.append(f1)

        plot_confusion_and_report(val_labels, val_preds, le, title=f"K-Fold␣
 ↪{fold+1}")

    # Print average metrics across folds
    print(f"\n=== Cross-Validation ({k}-fold) Summary ===")
    print(f"Average Accuracy: {np.mean(fold_accuracies):.4f}")
    print(f"Average Weighted F1: {np.mean(fold_f1_scores):.4f}")

# 4b. Time-Based Train/Test Split
def time_based_split(dataset, train_ratio=0.8, epochs=10):
    print("\n=== Time-Based Split ===")

    total_len = len(dataset)
    split_index = int(train_ratio * total_len)

    train_subset = Subset(dataset, range(split_index))
    test_subset  = Subset(dataset, range(split_index, total_len))
```

```
    train_loader = DataLoader(train_subset, batch_size=32, shuffle=False)
    test_loader  = DataLoader(test_subset, batch_size=32, shuffle=False)

    model = CNNPhaseClassifier(input_channels=len(numeric_columns),␣
 ↪num_classes=3)
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=1e-3)

    # Train
    for epoch in range(epochs):
        loss = train_one_epoch(model, train_loader, criterion, optimizer)

    # Evaluate
    test_labels, test_preds = evaluate_model(model, test_loader)
    acc = accuracy_score(test_labels, test_preds)
    f1 = f1_score(test_labels, test_preds, average="weighted")

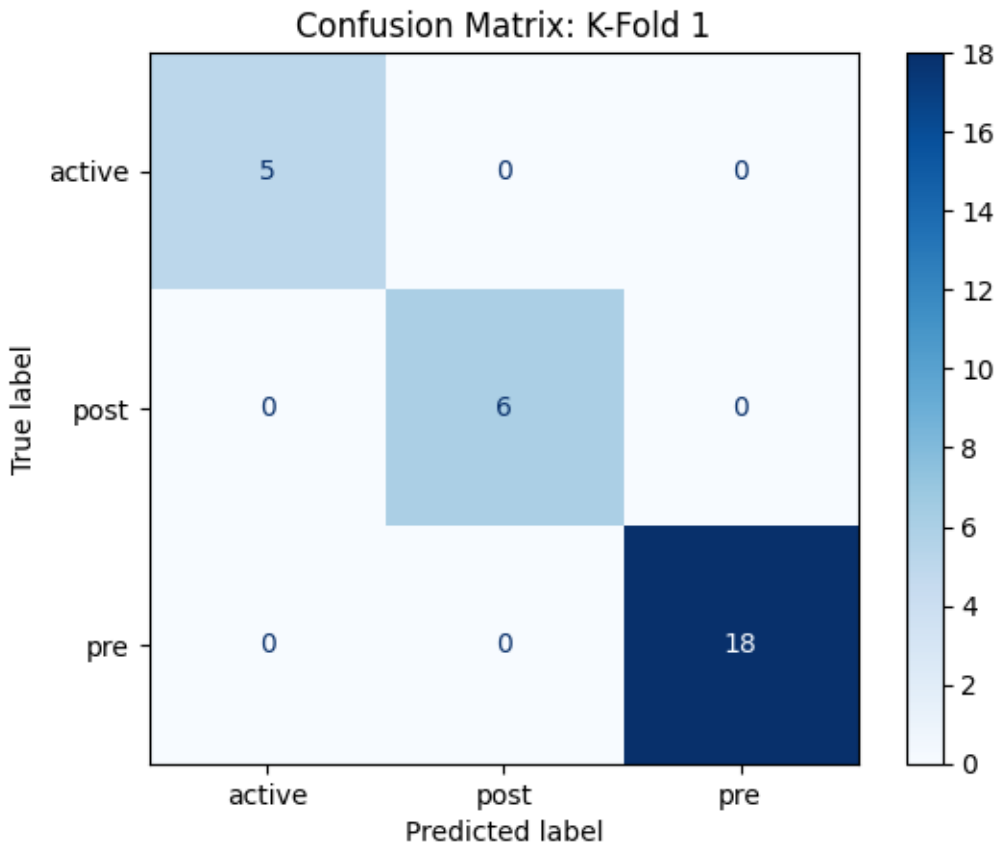
    print(f"Time-based Test - Accuracy: {acc:.4f}, Weighted F1: {f1:.4f}")

    # Confusion matrix + classification report
    plot_confusion_and_report(test_labels, test_preds, le, title="Time-based␣
 ↪Split")

# 5. Run All Approaches
if __name__ == "__main__":
    # 1) K-Fold Cross Validation
    cross_validate_model(dataset, k=5, epochs=10)

    # 2) Time-Based Train/Test Split
    time_based_split(dataset, train_ratio=0.8, epochs=10)
```

```
=== K-Fold: Fold 1/5 ===
Fold 1 - Accuracy: 1.0000, Weighted F1: 1.0000
```

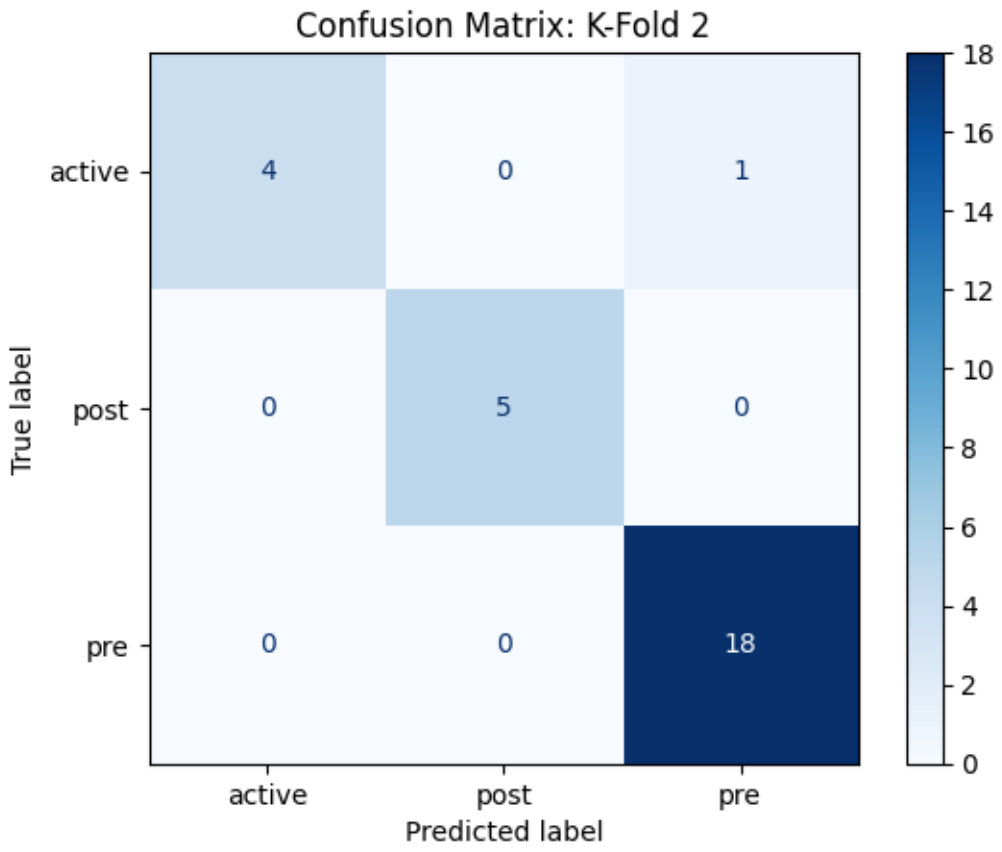



```
Classification Report: K-Fold 1
              precision    recall  f1-score   support

      active       1.00      1.00      1.00         5
        post       1.00      1.00      1.00         6
         pre       1.00      1.00      1.00        18

    accuracy                           1.00        29
   macro avg       1.00      1.00      1.00        29
weighted avg       1.00      1.00      1.00        29


=== K-Fold: Fold 2/5 ===
Fold 2 - Accuracy: 0.9643, Weighted F1: 0.9628
```

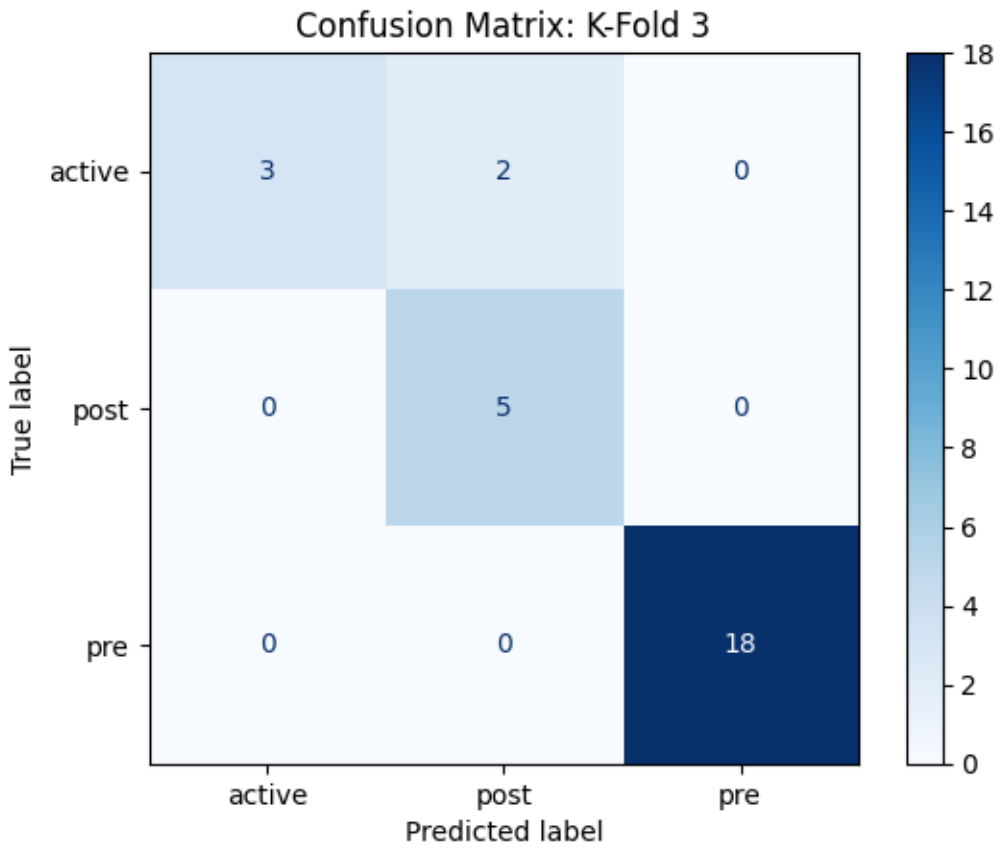



```
Classification Report: K-Fold 2
              precision    recall  f1-score   support

      active       1.00      0.80      0.89         5
        post       1.00      1.00      1.00         5
         pre       0.95      1.00      0.97        18

    accuracy                           0.96        28
   macro avg       0.98      0.93      0.95        28
weighted avg       0.97      0.96      0.96        28


=== K-Fold: Fold 3/5 ===
Fold 3 - Accuracy: 0.9286, Weighted F1: 0.9256
```

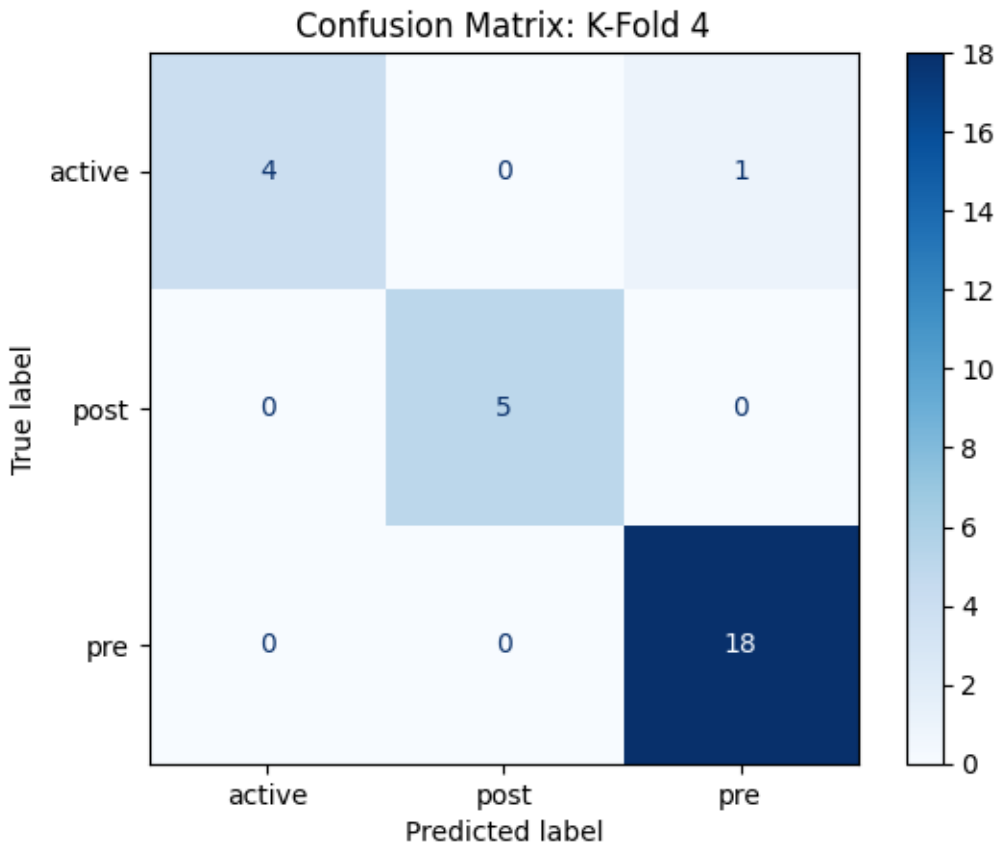



```
Classification Report: K-Fold 3
              precision    recall  f1-score   support

      active       1.00      0.60      0.75         5
        post       0.71      1.00      0.83         5
         pre       1.00      1.00      1.00        18

    accuracy                           0.93        28
   macro avg       0.90      0.87      0.86        28
weighted avg       0.95      0.93      0.93        28


=== K-Fold: Fold 4/5 ===
Fold 4 - Accuracy: 0.9643, Weighted F1: 0.9628
```

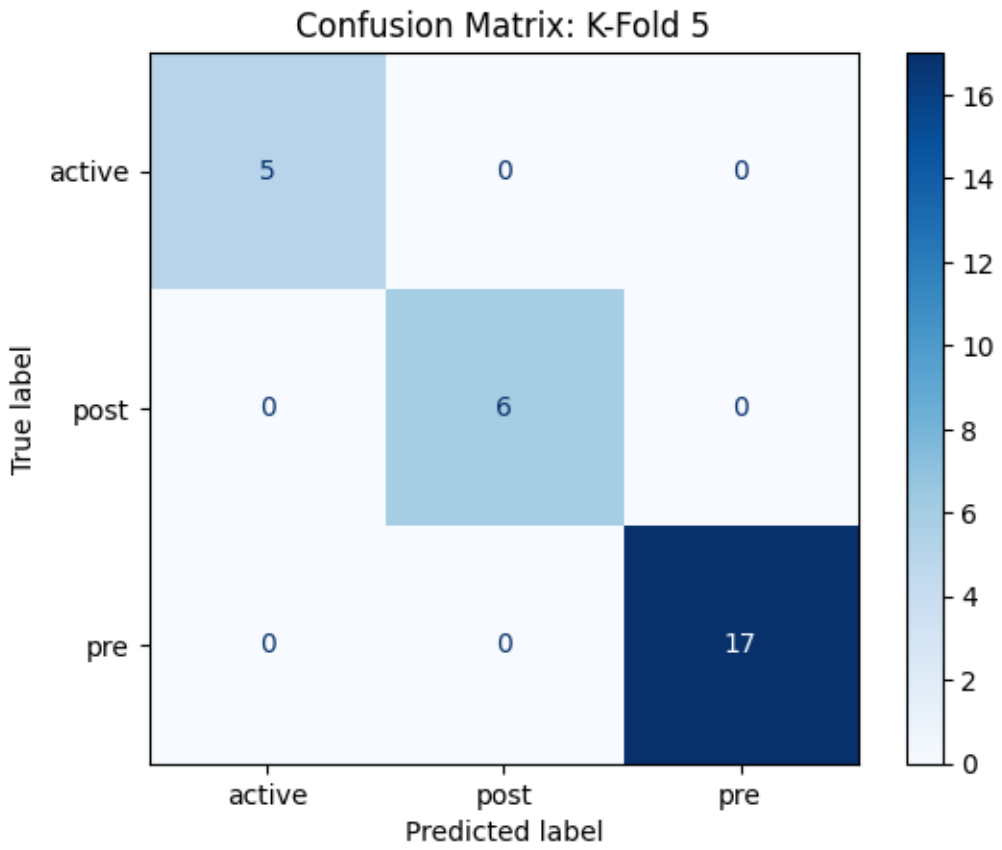



```
Classification Report: K-Fold 4
              precision    recall  f1-score   support

      active       1.00      0.80      0.89         5
        post       1.00      1.00      1.00         5
         pre       0.95      1.00      0.97        18

    accuracy                           0.96        28
   macro avg       0.98      0.93      0.95        28
weighted avg       0.97      0.96      0.96        28


=== K-Fold: Fold 5/5 ===
Fold 5 - Accuracy: 1.0000, Weighted F1: 1.0000
```



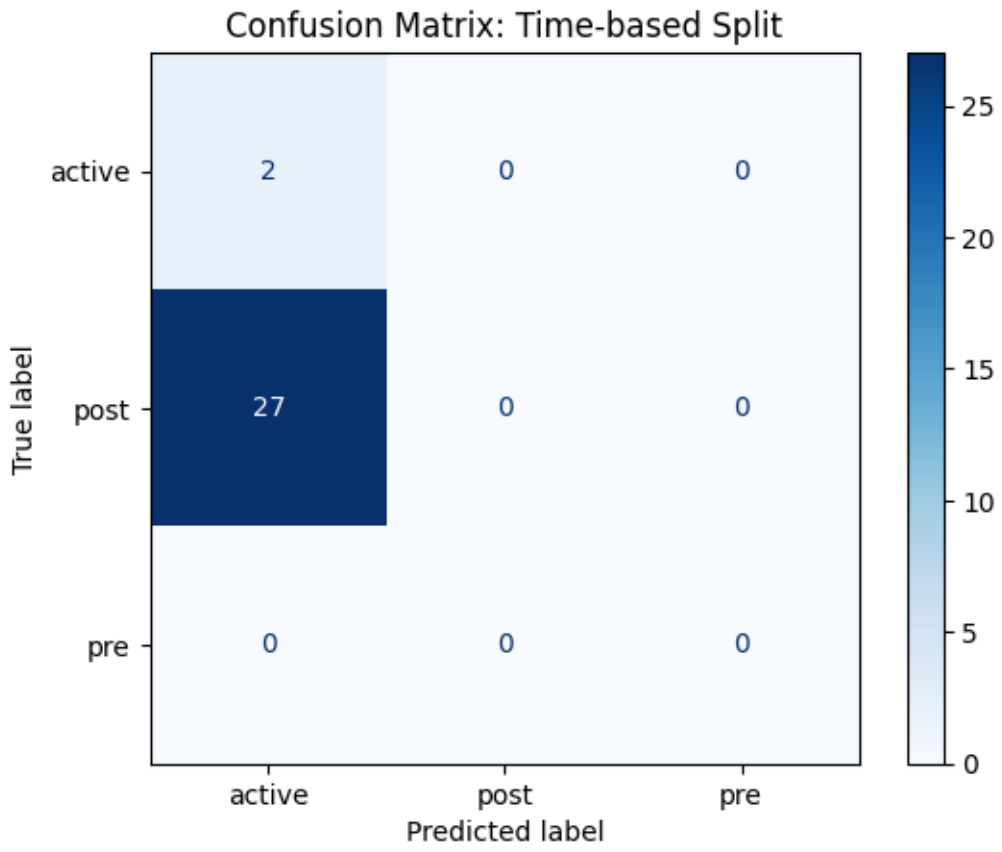

```
Classification Report: K-Fold 5
              precision    recall  f1-score   support

      active       1.00      1.00      1.00         5
        post       1.00      1.00      1.00         6
         pre       1.00      1.00      1.00        17

    accuracy                           1.00        28
   macro avg       1.00      1.00      1.00        28
weighted avg       1.00      1.00      1.00        28


=== Cross-Validation (5-fold) Summary ===
Average Accuracy: 0.9714
Average Weighted F1: 0.9702

=== Time-Based Split ===
Time-based Test - Accuracy: 0.0690, Weighted F1: 0.0089
```




```
Classification Report: Time-based Split
              precision    recall  f1-score   support

      active       0.07      1.00      0.13         2
        post       0.00      0.00      0.00        27
         pre       0.00      0.00      0.00         0

    accuracy                           0.07        29
   macro avg       0.02      0.33      0.04        29
weighted avg       0.00      0.07      0.01        29
```

```
/Users/jw/.pyenv/versions/ai894-group1-alt/lib/python3.13/site-
packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning:
Precision is ill-defined and being set to 0.0 in labels with no predicted
samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/Users/jw/.pyenv/versions/ai894-group1-alt/lib/python3.13/site-
packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Recall
is ill-defined and being set to 0.0 in labels with no true samples. Use
```

```
`zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/Users/jw/.pyenv/versions/ai894-group1-alt/lib/python3.13/site-
packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning:
F-score is ill-defined and being set to 0.0 in labels with no true nor predicted
samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/Users/jw/.pyenv/versions/ai894-group1-alt/lib/python3.13/site-
packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning:
Precision is ill-defined and being set to 0.0 in labels with no predicted
samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/Users/jw/.pyenv/versions/ai894-group1-alt/lib/python3.13/site-
packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Recall
is ill-defined and being set to 0.0 in labels with no true samples. Use
`zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/Users/jw/.pyenv/versions/ai894-group1-alt/lib/python3.13/site-
packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning:
F-score is ill-defined and being set to 0.0 in labels with no true nor predicted
samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/Users/jw/.pyenv/versions/ai894-group1-alt/lib/python3.13/site-
packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning:
Precision is ill-defined and being set to 0.0 in labels with no predicted
samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/Users/jw/.pyenv/versions/ai894-group1-alt/lib/python3.13/site-
packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Recall
is ill-defined and being set to 0.0 in labels with no true samples. Use
`zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/Users/jw/.pyenv/versions/ai894-group1-alt/lib/python3.13/site-
packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning:
F-score is ill-defined and being set to 0.0 in labels with no true nor predicted
samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```